informacion.java
Archivo

Main(1).java
Archivo

sabes que necesito que despues de que se selccione el personaje, abra la pantalla de informacion antes de caer en el mapa principal. Pero asi como esta ahora despues del menu de seleccion, cae directo es en el primer nivel

```
package com.juego.progra;

import com.badlogic.gdx.Gdx;
import com.badlogic.gdx.graphics.GL20;
import com.badlogic.gdx.graphics.Pixmap;
import com.badlogic.gdx.graphics.Texture;
import com.badlogic.gdx.graphics.g2d.SpriteBatch;
```

Ver más

Sí. Ya viendo tu `Main.java` y `informacion.java`, encontré **el problema principal**. No es realmente la `Transicion`: es la forma en que estás manejando `informacion`.

Qué está pasando ahora

Tu flujo actual es:

```
MENÚ
  ↓
SELECCIÓN DE PERSONAJE
  ↓
PRIMER NIVEL  ← esto es lo que te está pasando
```

Y tú quieres:

```
MENÚ
  ↓
SELECCIÓN DE PERSONAJE
  ↓
INFORMACIÓN
  ↓
MAPA PRINCIPAL
  ↓
PRIMER NIVEL
```

En tu `Main`, cuando seleccionas el personaje haces esto:

```
Java

personajeElegido = (elegido == SeleccionPersonaje.Opcion.PERSONAJE1) ? 1 : 2;
```

```
enSeleccion = false;
transicion.empezarPantalla(-6);
```

Eso está bien: `-6` debería llevarte a información. De hecho, más abajo tienes:

Java

```
}else if(destino==-6){
    setScreen(new informacion(this));
}
```

Main

Pero aquí está el problema de diseño: `informacion` es un `Screen` independiente:

Java

```
public class informacion implements Screen
```

y cuando llegas ahí haces:

Java

```
setScreen(new informacion(this));
```

Eso significa que **Main deja de ser la pantalla activa**. La pantalla activa pasa a ser `informacion`. informacion

Luego, cuando presionas el botón de información, haces:

Java

```
main.irAlMapaPrincipal();
```

pero `irAlMapaPrincipal()` solamente cambia variables de `Main`; **no vuelve a poner a Main como pantalla activa**. informacion

---

La solución que te recomiendo

Como tu juego ya está construido alrededor de `Main` controlando:

- menú
- selección
- mapa principal
- niveles

- transición

no vamos a convertir `informacion` en otro `Screen`.

Vamos a tratar la información como **otro estado del juego**, exactamente igual que `enMenu`, `enSeleccion` y `enMapaPrincipal`.

Así queda mucho más limpio:

```
Main
 ├── enMenu
 ├── enSeleccion
 ├── enInformacion   ← NUEVO
 ├── enMapaPrincipal
 └── nivel
```

Y no tienes que rehacer tu sistema de transición.

---

1. Primero modifica `Main`

Actualmente tienes:

```java
private boolean enMenu = true;
private SeleccionPersonaje seleccionPersonaje;
private boolean enSeleccion = false;
```

Déjalo así y agrega:

```java
private boolean enInformacion = false;
private informacion pantallaInformacion;
```

Quedaría:

```java
private Menu menu;
private boolean enMenu = true;

private SeleccionPersonaje seleccionPersonaje;
private boolean enSeleccion = false;

private informacion pantallaInformacion;
```

```
private boolean enInformacion = false;

private int personajeElegido = 2;
```

---

2. Créala en `create()`

Actualmente tienes:

```
Java

seleccionPersonaje = new SeleccionPersonaje();
HUD = new HUD(this);
```

Agrega:

```
Java

pantallaInformacion = new informacion(this);
```

Pero **ojo**: para hacer esto tendremos que modificar `informacion.java`, porque actualmente su constructor crea su propio `Stage` y funciona como un `Screen`.

---

3. Cambiemos `informacion.java`

No necesitamos que `informacion` implemente `Screen`.

Cambia:

```
Java

public class informacion implements Screen {
```

por:

```
Java

public class informacion {
```

Y conserva:

```
Java

private Main main;
private Stage stage;
```

```
private SpriteBatch batch;
```

Pero como ahora `Main` será quien controle cuándo se muestra, vamos a darle métodos para actualizar y dibujar.

Tu constructor puede quedarse prácticamente igual:

```java
public informacion(Main main) {
    this.main = main;
    this.stage = new Stage();
    this.batch = new SpriteBatch();

    historiaPrincipio();
    botonesPrincipio();
}
```

Pero **quita**:

```java
Gdx.input.setInputProcessor(stage);
```

porque ahora `Main` tendrá que controlar el input.

---

4. Cambia el botón de información

Ahora mismo tienes esto:

```java
} else if (pantallaAct == 2) {
    main.irAlMapaPrincipal();
}
```

Eso está bien conceptualmente, pero ahora queremos que `Main` cambie de estado.

Déjalo así:

```java
} else if (pantallaAct == 2) {
    main.irAlMapaPrincipal();
}
```

No hace falta cambiarlo todavía.

5. Agrega métodos a `informacion`

En lugar de tener:

Java

```
@Override
public void render(float delta)
```

vamos a tener:

Java

```
public void actualizar(float delta) {
    stateTime += delta;
    stage.act(delta);
}
```

Y:

Java

```
public void dibujar() {
    ScreenUtils.clear(0, 0, 0, 1);

    batch.begin();

    if (pantallaAct == 1) {
        batch.draw(p1, 0, 0,
                Gdx.graphics.getWidth(),
                Gdx.graphics.getHeight());
    } else {
        batch.draw(p2, 0, 0,
                Gdx.graphics.getWidth(),
                Gdx.graphics.getHeight());
    }

    dibujarRector(batch, stateTime);

    batch.end();

    stage.draw();
}
```

Entonces `informacion` deja de tener:

Java

```
@Override
public void render(float delta)
```

porque **Main será quien la dibuje**.

---

6. Ahora viene el cambio importante en `Main`

En tu código tienes:

```
Java

}else if(destino==-6){
    setScreen(new informacion(this));
}
```

ELIMINA ESO.

Y reemplázalo por:

```
Java

}else if(destino == -6){
    enInformacion = true;
    enMenu = false;
    enSeleccion = false;
    enMapaPrincipal = false;
}
```

Ahora la transición hará:

```
Selección
   ↓
transicion -6
   ↓
enInformacion = true
```

Sin abandonar `Main`.

---

7. Modifica el bloque de selección

Actualmente tienes:

```
Java

} else if (enSeleccion) {
```

```
    SeleccionPersonaje.Opcion elegido = seleccionPersonaje.actualizar();

    if (seleccionPersonaje.huboClic())
        musicaAmbiente.agachar();

    if (elegido != SeleccionPersonaje.Opcion.NINGUNA) {
        personajeElegido =
            (elegido == SeleccionPersonaje.Opcion.PERSONAJE1) ? 1 : 2;

        Gdx.app.log("SELECCION", "Personaje elegido: " + elegido);

        enSeleccion = false;
        transicion.empezarPantalla(-6);
    }
}
```

Eso **puede quedarse exactamente igual**.

Y eso es bueno, porque ya tienes perfectamente hecha la transición.

---

8. Agrega el estado `enInformacion` en `render()`

Tienes:

```
    Java

if (!enTransicion) {
    if (enMenu) {
        ...
    } else if (enSeleccion) {
        ...
    } else if (enMapaPrincipal) {
        ...
    } else {
        ...
    }
}
```

Agrega `enInformacion` entre selección y mapa:

```
    Java

if (!enTransicion) {

    if (enMenu) {

        Menu.Opcion elegida = menu.actualizar();

        if (menu.huboClic())
```

```
            musicaAmbiente.agachar();

        if (elegida == Menu.Opcion.JUGAR) {
            transicion.empezarPantalla(-1);
        } else if (elegida == Menu.Opcion.SALIR) {
            Gdx.app.exit();
        }

    } else if (enSeleccion) {

        SeleccionPersonaje.Opcion elegido =
                seleccionPersonaje.actualizar();

        if (seleccionPersonaje.huboClic())
            musicaAmbiente.agachar();

        if (elegido != SeleccionPersonaje.Opcion.NINGUNA) {

            personajeElegido =
                    (elegido == SeleccionPersonaje.Opcion.PERSONAJE1)
                            ? 1 : 2;

            Gdx.app.log(
                    "SELECCION",
                    "Personaje elegido: " + elegido
            );

            enSeleccion = false;

            transicion.empezarPantalla(-6);
        }

    } else if (enInformacion) {

        pantallaInformacion.actualizar(delta);

    } else if (enMapaPrincipal) {

        // tu código actual del mapa...

    } else {

        // tu código actual del nivel...
    }
}
```

---

9. Ahora modifica `dibujarEscena()`

Actualmente tienes:

Java

```
if (enMenu) {
    ...
} else if (enSeleccion) {
    ...
} else if (enMapaPrincipal) {
    ...
} else {
    ...
}
```

Agrega información:

Java

```
private void dibujarEscena() {

    if (enMenu) {

        batch.begin();
        menu.dibujar(batch);
        batch.end();

    } else if (enSeleccion) {

        batch.begin();
        seleccionPersonaje.dibujar(batch);
        batch.end();

    } else if (enInformacion) {

        pantallaInformacion.dibujar();

    } else if (enMapaPrincipal) {

        // TODO tu código actual del mapa

    } else {

        // TODO tu código actual del nivel
    }
}
```

---

10. Y ahora `irAlMapaPrincipal()` queda perfecto

Tu método actual es:

Java

```java
public void irAlMapaPrincipal() {
    enMapaPrincipal = true;
    enMenu=false;

    for (PistaMusical p : musicasNiveles) {
        p.detener();
    }

    if (!musicaAmbiente.estaSonando()) {
        musicaAmbiente.empezar();
    }

    nuevoJugador(
        mapaPrincipal.getSpawnX(),
        mapaPrincipal.getSpawnY()
    );

    mapaPrincipal.mostrarBotonMenu(true);
}
```

Main

Hay que agregar:

Java

```java
enInformacion = false;
enSeleccion = false;
```

Entonces:

Java

```java
public void irAlMapaPrincipal() {

    enInformacion = false;
    enSeleccion = false;
    enMenu = false;
    enMapaPrincipal = true;

    for (PistaMusical p : musicasNiveles) {
        p.detener();
    }

    if (!musicaAmbiente.estaSonando()) {
        musicaAmbiente.empezar();
    }
```

```
        nuevoJugador(
            mapaPrincipal.getSpawnX(),
            mapaPrincipal.getSpawnY()
        );

        mapaPrincipal.mostrarBotonMenu(true);
    }
```

Hay otro detalle MUY importante

En tu `create()` tienes esto:

```
Java

cambiarNivel(1);
```

Main

Eso significa que **el nivel 1 se carga desde el principio**, aunque el jugador todavía esté en el menú.

Eso no necesariamente es un problema porque:

```
Java

private boolean enMapaPrincipal = true;
```

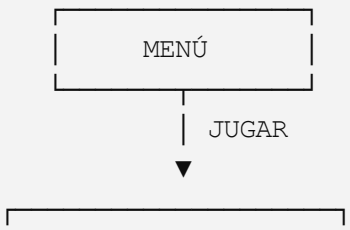
y después el juego empieza mostrando el mapa principal.

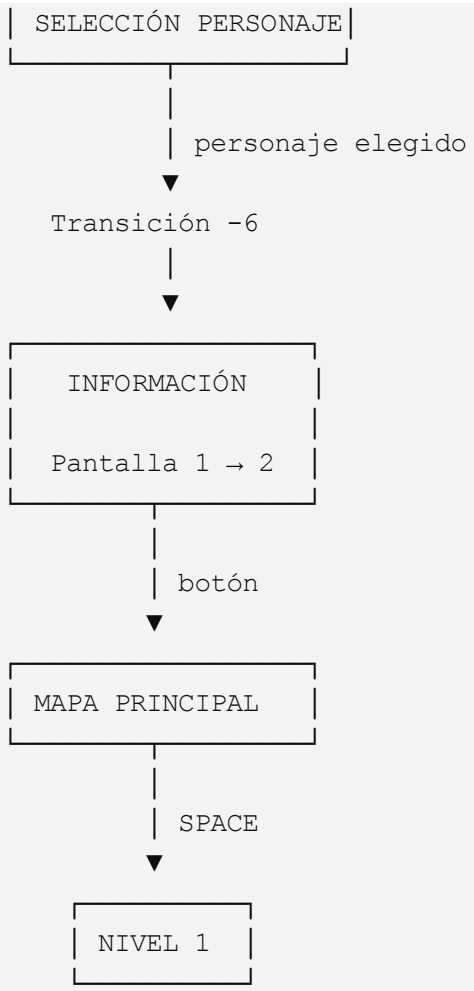
Pero si quieres que el juego empiece realmente limpio, podemos posteriormente cambiar la inicialización.

No te recomiendo tocarlo todavía, porque primero hagamos funcionar correctamente el flujo.

El flujo final quedaría así

Con los cambios anteriores:





Y lo más importante: **no vamos a usar** `setScreen(new informacion(this))`. Tu `Main` seguirá siendo el controlador de todo el juego. Esto además evita que después tengas problemas para regresar desde información al mapa.

Tu sistema de estados ya está bastante bien armado; solamente estabas mezclando **dos arquitecturas distintas**: estados dentro de `Main` (`enMenu`, `enSeleccion`, `enMapaPrincipal`) y una `Screen` independiente (`informacion`). La primera es la que te conviene seguir.

Fuentes

ChatGPT puede cometer errores. Se debe comprobar la información importante